Back to CCDV-F Sample Tests

Great Job!

Agents and Workflows Practice Test Complete

8/10	80%	6:23
Correct	Score	Time

This was a sample test from a limited pool of questions.
Upgrade to Premium for the complete CCDV-F bank with hundreds more questions!

Review Your Answers

1

Login to Clarify

A team is automating monthly expense report handling with Claude. Every report must follow the same

sequence: extract fields, check policy rules, route exceptions to a human queue, and generate a summary. The branching conditions are fixed and success is easy to verify. Which architecture is the best fit?

A.
A deterministic workflow that calls Claude only at defined steps

B.
A fully autonomous agent that decides each next action

C.
A manager agent that delegates every report to subagents

D.
A long-running agent that stores every report interaction

Explanation:

Correct answer (A): A deterministic workflow is the best fit because the process has a known sequence, predictable branching, and clear success criteria. Claude can still be used inside defined steps, but the orchestration should not be left to an autonomous agent when the application already knows the correct order and routing logic.

Why the other options are wrong:

- Option B: An autonomous agent may appear flexible, but the task does not require dynamic planning or open-ended tool choice. It would add unnecessary variability and complexity.
- Option C: Subagents may help with specialized decomposition, but this scenario is a simple fixed process. A manager/subagent pattern would add coordination overhead without a clear benefit.
- Option D: Long-running memory is not needed for a predictable monthly workflow. Keeping every interaction would risk context bloat without improving the fixed routing

process.

2

Login to Clarify

A developer is building a Claude-powered internal research assistant. User requests vary widely, and the assistant must decide whether to search documentation, inspect previous notes, ask a clarifying question, synthesize findings, or stop based on intermediate results. Which design is most appropriate?

A.
A fixed workflow with the same steps for every request

B.
An agent loop that plans, acts, observes, and stops

C.
A batch job that summarizes all notes nightly

D.
A single prompt that forbids asking clarifying questions

Explanation:

Correct answer (B): An agent loop is appropriate because the task requires dynamic planning, choosing among actions, reacting to intermediate observations, and deciding when to ask for clarification or stop. These choices cannot be fully enumerated as a fixed sequence for every research request.

Why the other options are wrong:

- Option A: A fixed workflow may seem easier to operate, but it is too rigid for requests that

require different paths based on intermediate findings.

- Option C: A nightly batch summary may be useful for a separate reporting use case, but it does not support interactive planning and adaptive research.
- Option D: Forbidding clarifying questions removes a useful agent behavior. The scenario explicitly includes cases where clarification may be the right next action.

3

Login to Clarify

A Claude-powered support agent repeatedly checks the same ticket metadata tool after receiving unchanged results. The application currently lets the model decide whether to continue, with no retry limit or completion criteria. What is the best primary change?

A. Add explicit stop conditions and retry limits to the loop

B. Increase the conversation history kept for each ticket

C. Split every ticket into several subagent assignments

D. Replace all tool calls with a fixed daily report

Explanation:

Correct answer (A): Agent loops should have explicit stopping conditions, retry limits, and clear handling for repeated or unhelpful tool results. This prevents runaway loops and unnecessary repeated attempts when the agent is not making progress.

Why the other options are wrong:

- Option B: More history may make the loop even larger without addressing the missing termination logic that is causing repeated tool checks.
- Option C: Subagents can help with decomposition, but the immediate flaw is a loop-control problem, not a lack of specialized workers.
- Option D: A fixed daily report avoids the loop but does not satisfy the interactive support-agent requirement described in the scenario.

4

Login to Clarify

A Claude-powered due diligence assistant reviews large acquisition packets. One part analyzes financial tables, another checks contract obligations, and another summarizes technical risks. A single general agent is mixing evidence across domains and exceeding its useful context. What is the best architecture change?

A. Keep one agent and append all evidence to its memory

B. Use a manager agent with specialized subagents per domain

C. Convert the process into one fixed linear extraction workflow

D. Ask the general agent to be more careful about sources

Explanation:

Correct answer (B): A manager/subagent pattern is appropriate when a complex task decomposes into specialized subtasks. Specialized subagents can receive narrower context and tool access, reducing evidence mixing and improving context isolation while a manager coordinates the overall result.

Why the other options are wrong:

- Option A: Appending all evidence worsens context pressure and does not address cross-domain confusion. It reinforces the single overloaded-agent failure mode.
- Option C: A fixed linear workflow may work for predictable extraction, but the scenario involves specialized analysis and synthesis across complex domains.
- Option D: Better instructions may help slightly, but they do not solve the architectural issue of one agent handling too much unrelated context.

5

Login to Clarify

A recruiting assistant uses an agent to screen incoming candidate messages. After the agent classifies a message as needing human review, the application must always create a task in the HR system at that exact point. Which implementation pattern is best?

A.
Tell the model to remember to create the HR task

B.
Use a deterministic hook after the review classification

C.
Let the agent choose whether task creation is needed

D.
Store the decision in memory for the next conversation

Explanation:

Correct answer (B): Hooks are appropriate when a specific action must occur deterministically at a defined point in an agent workflow. The model can classify the message, but the application should enforce the required side effect instead of relying only on model discretion.

Why the other options are wrong:

- Option A: Prompting the model may appear simple, but instructions alone do not make the side effect deterministic at a specific point in the workflow.
- Option C: The scenario says the task must always be created after the classification. Leaving the decision to the agent conflicts with that requirement.
- Option D: Memory can preserve useful state, but saving the decision for later does not perform the required immediate workflow action.

6

Login to Clarify

A startup is building its first Claude-powered operations agent. The team wants supported agent-building abstractions for loop handling and tool dispatch, and it does not need custom state-machine behavior yet. Which implementation choice is most appropriate?

A.
Use the Claude Agent SDK for the initial agent

B.
Build a fully custom orchestration harness first

C.
Avoid an agent and use a static prompt only

D.
Create subagents before defining the main task

Explanation:

Correct answer (A): The Claude Agent SDK is appropriate when the team wants supported abstractions instead of implementing loop, state, and tool-dispatch behavior manually. A custom harness is better reserved for cases needing tight orchestration control or specialized integration behavior.

Why the other options are wrong:

- Option B: A custom harness can be valuable, but the scenario says the team wants supported abstractions and lacks special state-machine requirements.
- Option C: A static prompt does not provide the agent loop and tool-dispatch behavior the team is explicitly seeking.
- Option D: Subagents should be introduced to solve decomposition or context-isolation needs, not before the main task and architecture are defined.

7

Login to Clarify

A long-running Claude agent helps project managers across many weeks. Quality declines because old tool outputs, resolved decisions, and unrelated chat history are repeatedly included in the context. The team still needs durable project facts for future decisions. What is the best primary change?

A.
Keep all prior messages so the agent never loses information

B.
Summarize durable facts and prune irrelevant history

C.
Restart the agent after each message with no memory

D.
Add subagents for every individual project message

Explanation:

Correct answer (B): Agent memory should preserve useful information while avoiding unbounded accumulation of irrelevant messages and stale tool outputs. Summarizing durable facts and pruning unrelated history reduces context bloat and context drift while retaining state needed for future decisions.

Why the other options are wrong:

- Option A: Keeping everything may appear safest, but it is the source of context bloat and stale information in this scenario.
- Option C: Removing all memory avoids bloat but loses durable project facts that the team explicitly needs for future decisions.
- Option D: Subagents can isolate specialized work, but creating one for every message adds overhead and does not define a memory strategy.

8

Login to Clarify

An enterprise platform team is deploying a Claude-powered remediation agent. It must integrate with an existing internal workflow engine, emit step-level traces into the company observability stack, and use custom state transitions for retries and escalations. The team can operate its own runtime. Which

deployment approach best matches these requirements?

A. Use a self-hosted custom agent loop or harness

B. Use managed deployment to avoid all runtime decisions

C. Use a simple deterministic workflow with no agent loop

D. Use one general agent with unbounded conversation memory

Explanation:

Correct answer (A): A self-hosted custom loop or harness is the best fit when the application requires tight control over orchestration, state transitions, observability, retries, and integration with existing infrastructure. The team is also able to operate the runtime, which supports this choice.

Why the other options are wrong:

- Option B: Managed deployment can reduce operational burden, but the scenario emphasizes custom orchestration, internal observability, and state-transition control.
- Option C: A deterministic workflow may be appropriate for fully predictable processes, but this scenario specifies a remediation agent with custom retry and escalation behavior.
- Option D: Unbounded memory is not a deployment strategy and would risk context bloat rather than meeting observability and orchestration requirements.

9

Login to Clarify

A finance team is automating vendor invoice handling with Claude. The process is predictable and audited weekly. Artifact excerpt:

Runbook:

1. Extract invoice fields from PDF text.
2. If amount < $5,000 and vendor is approved, create payment draft.
3. If amount >= $5,000, route to manager approval.
4. If tax ID is missing, reject with a fixed reason.
5. Log each branch decision.

Which architecture is the best fit for this automation?

A.
A free-form agent that decides the next step after each invoice

B.
A deterministic workflow with Claude used inside fixed steps

C.
A manager agent that delegates every invoice to subagents

D.
A memory-heavy agent that learns approvals from prior invoices

Explanation:

Correct answer (B): A deterministic workflow is best when the sequence of steps, branching rules, and audit requirements are stable. Claude can still be used inside bounded steps, such as

extracting fields or drafting messages, while the application enforces approval routing and logging deterministically.

Why the other options are wrong:

- Option A: A free-form agent may seem flexible, but the process does not require dynamic planning and would be harder to audit than fixed branching logic.
- Option C: A manager/subagent design adds coordination overhead without a need for specialized decomposition or context isolation.
- Option D: Memory from prior invoices is not the primary need, and storing approval history in agent memory could weaken auditability.

10

Login to Clarify

A product team wants Claude to investigate intermittent customer complaints. The path varies by case: it may need to inspect release notes, query incident summaries, compare support tickets, and decide which evidence is relevant. Artifact excerpt:

Task sample:

- Complaint: "Mobile checkout fails after coupon entry."
- Possible sources: ticket search, incident tracker, release notes, metrics summary
- Required output: likely cause, evidence links, uncertainty notes
- Search path is not known before the task starts.

Which implementation pattern is most appropriate?

A. A fixed workflow that queries every source in a preset order

B.
An agent loop with planning, tool actions, observations, and stopping criteria

C.
A static prompt that asks Claude to infer the cause without tools

D.
A memory-only assistant that relies on previously stored incidents

Explanation:

Correct answer (B): An agentic loop fits because the task requires dynamic planning, adaptive tool selection, and iterative reasoning over observations. The loop should include state, model decisions, tool actions, observations, and explicit stopping or escalation criteria.

Why the other options are wrong:

- Option A: A fixed workflow may be easier to test, but it is inefficient and brittle when the relevant evidence path differs across cases.
- Option C: A static prompt avoids orchestration, but the scenario explicitly requires consulting changing external sources.
- Option D: Memory can help with durable context, but relying only on prior stored incidents ignores case-specific investigation.